

Sharded Inference of a 229B-Parameter MoE over the Public Internet at Interactive Speed

WAN-aware speculative decoding with verifiable execution on scattered consumer GPUs

leyten — c0mpute

July 2026 · code, receipts and raw measurement JSONs: github.com/leyten/shard

Abstract. We present a working system that serves MiniMax-M2.5 — a 229B-parameter mixture-of-experts model (10B active, 62 layers, 115 GB in NVFP4) — sharded across five consumer RTX 5090s in five European countries, connected only by the public internet over an encrypted libp2p transport. Single-stream decoding sustains 10–13 tokens/s on interactive reasoning workloads and 70–87 tokens/s on draftable text, with every request accompanied by per-stage cryptographic execution receipts. Three results make this possible and are, to our knowledge, novel at this scale over real wide-area networks: (1) an **accept-gated pipelining law** for speculative decoding over high-latency links — pipelined speculation collapses below a per-token acceptance rate of $\alpha \approx 0.80$, which no drafter reaches on novel text — and the **depth-aware hybrid** coordinator this law forces, which pipelines draftable spans and serializes novel ones; (2) the first per-stage transport/compute decomposition measured on a live scattered ring, which shows stage time on consumer fleets is dominated by **CPU kernel-launch overhead**, not GPU throughput; (3) a verification layer — signed per-stage activation hash-chains with fail-closed layer-coverage checks — whose measured overhead is below 1% of stage compute (0.05 ms on an 11.7 ms stage span). We report a full, reproducible evaluation: interleaved multi-arm benchmarks with repetitions, batched aggregate throughput of 150–194 tokens/s (B=4; 155 tokens/s at 16k context on a six-stage ring), and live fault-recovery timelines. All numbers link to signed receipts in the public repository.

1 Introduction

The premise of c0mpute is that the world’s idle consumer GPUs can form a single permissionless fabric for large-model compute — BitTorrent, but for VRAM and FLOPs rather than disk. The load-bearing question for that thesis has always been concrete: can a frontier-scale model, sharded across GPUs that share nothing but the public internet, serve a single user at a speed that feels usable — and can the user **verify** they got what they paid for?

This report answers both questions with a running system and signed measurements. Our engine (**shard**) splits MiniMax-M2.5 — 229B total parameters, 10B active per token, 62 transformer layers, 115 GB of NVFP4 weights — across five RTX 5090 stages rented from independent hosts in Czechia, Switzerland, Norway and Denmark, with inter-stage round-trips of 5–38 ms and no co-located pair of stages. On this ring the system sustains 10–13 tokens/s of single-stream novel reasoning with signed receipts enabled, reaching 70–87 tokens/s on draftable spans and 150–194 tokens/s aggregate under B=4 batching.

Contributions, each backed by receipts in the public repository:

1. **A WAN-native speculative decoding stack and the analysis that shapes it.** We derive and empirically validate the accept-gated pipelining law (§4): a depth-D speculative pipeline only pays when the prior chunk fully accepts, an event with probability α^K that collapses below $\alpha \approx 0.80$ — a threshold no current drafter clears on novel text (EAGLE-3, which conditions on the target model’s own hidden state, measures $\alpha \approx 0.74$). The law explains why the “obvious” fix for WAN latency — pipelining — cannot work for reasoning workloads, and forces the design we ship: a **depth-aware hybrid** that routes each round at runtime, pipelining n-gram-draftable spans at depth while verifying novel spans as speculation **trees** in single synchronous rounds.
2. **A measured anatomy of WAN inference on consumer hardware.** Per-stage timing instrumentation decomposes every traversal into transport and per-stage compute. Two findings re-rank the

optimization landscape: transport (wire + relay + codec) accounts for 55–68% of a traversal on a well-selected ring, and the **compute** term is bounded not by the GPU but by the host CPU’s single-thread kernel-launch rate — identical GPUs differ 4× in stage time depending on the CPU and co-tenant load behind them (§5.3).

3. **Verifiable execution at negligible cost.** Every stage signs an activation hash-chain over its layer range; the coordinator verifies signatures and **fail-closed** coverage of all 62 layers against the model’s own configuration, so a stage cannot skip work and still be paid. Measured end-to-end overhead: below 1% of stage compute (0.05 ms on an 11.7 ms stage span) (§5.5).
4. **Operational results a permissionless network needs.** Measured topology selection (RTT, VRAM, uplink, and CPU-speed probes feeding a ring optimizer with deployable-orientation guarantees), and live fault tolerance: a coordinator killed mid-decode is replaced without re-warming the ring, and a killed stage triggers cascade re-handshake on warm weights (§5.6).

We position these results against the published state of the art in §2 — including the system we consider the closest published measurement, a two-GPU LAN study of sharded serving [1] — and discuss honestly what a single stream over WAN cannot do (§6): the same analysis that yields our design also proves a physics ceiling for serial novel-text decoding, and we report how close to it we operate.

2 Related work

Distributed inference over volunteer/consumer hardware. Petals [2] demonstrated collaborative inference of BLOOM-176B over volunteer GPUs, reporting on the order of 1 token/s for single-stream generation over true wide-area links — the published reference point for frontier-scale WAN inference. Our setting is stricter (no trusted volunteers; every stage must **prove** its work) and our single-stream results are roughly an order of magnitude faster at comparable model scale, on cheaper hardware.

LAN sharding studies. A recent industry report [1] benchmarks pipeline- and tensor-parallel sharding of 3B–70B dense and 30B MoE models across two RTX A6000s on a sub-millisecond, 5 Gb/s link, and projects degradation curves for higher-latency settings. Its conclusions — pipeline parallelism dominates tensor parallelism off-LAN; MoE architectures are communication-efficient per parameter; speculative decoding is the promising next step — agree with our measurements where they overlap. The present report differs in kind rather than degree: we operate a 229B MoE on **measured** 5–38 ms public-internet links (not projections), with speculative decoding built and characterized rather than proposed, and with a verification layer absent from prior systems. Fault-tolerant sharded serving, listed there as an open goal, is demonstrated live in §5.6.

Speculative decoding. Speculative sampling [3], [4] accelerates decoding by verifying cheap draft tokens in parallel; Medusa [5] and EAGLE [6] refine the drafter, with EAGLE-3 [7] conditioning a lightweight head on the target model’s hidden states — the strongest published per-token acceptance on novel text ($\alpha \approx 0.74$ on reasoning workloads). All published spec-decode systems assume drafter and verifier share a device or a datacenter interconnect. Over WAN the economics invert — verification costs a full network round-trip — which is precisely the regime §4 analyzes. Tree-structured verification (SpecInfer [8], EAGLE-2/3) packs more candidates per verification; our contribution is not the tree but the **routing law** that decides, per round, between pipelined linear speculation and synchronous tree speculation.

Verifiable computation. Full cryptographic verification of LLM inference (ZK/KZG commitments) remains impractical at 100B+ scale. Our receipts implement the economic middle ground: signed activation hash-chains binding each stage to its exact layer range and activations, verified against the model’s true depth, fail-closed. This makes free-riding detectable at the cost of a hash and a signature per stage per request.

3 System

3.1 Model and placement

MiniMax-M2.5 is a 229B-parameter mixture-of-experts transformer (62 layers, GQA attention, 10B active parameters per token) quantized to NVFP4 experts with bf16 attention — 115 GB of weights. Each ring stage holds a contiguous layer block sized to its measured free VRAM (10–13 layers on a 32 GB RTX 5090, weights + KV within 30 GB), so the model that fits on no single consumer GPU fits comfortably on five. Pipeline parallelism is the only parallelism that survives WAN latency (consistent with [1]’s LAN measurements): a stage forwards a single small activation tensor — $[\text{tokens} \times 3072]$ at fp8, a few tens of kilobytes — per verification round, rather than per-layer all-reduces.

3.2 Transport and topology

Stages communicate through per-box libp2p sidecars (Noise-encrypted, NAT-traversing), each stage dialing only its successor; the tail returns results directly to the coordinator. Frames use a compact self-describing binary codec (JSON header + raw tensor payloads, no pickling), with activations and drafter side-channel tensors quantized to fp8 for the wire.

Ring composition is **measured, not assumed**: before weights are placed, the launcher probes the candidate pool — all-pairs TCP RTT, free VRAM, /24 subnet (co-location exclusion), uplink bandwidth, and single-thread CPU speed with load (a consequence of §5.3) — and a pure combinatorial optimizer (**shard/topology.py**) selects the subset, ring order, and per-node layer blocks that minimize predicted step time, with the coordinator’s stage pinned first so the chosen ring is exactly the ring that launches. Layer blocks bake into per-node weight downloads only after this decision.

3.3 Coordinator, drafters, and the serving surface

A single coordinator process (on the head box, co-located with stage 0) drives prefill and decode, runs the drafter stack (§4), commits verified tokens, and exposes an OpenAI-compatible `/v1/chat/completions` endpoint with streaming, tool-calling and multi-turn support. Prefill is chunked and pipelined depth-8 through the ring; decode framing is decided per round by the hybrid router. Greedy decoding is **lossless by construction**: the ring’s own argmax decides every committed token; speculation only changes how many candidate tokens each round-trip evaluates.

3.4 Verifiable execution receipts

Every stage maintains a per-request hash-chain over its (input, output) activation pairs and signs it, together with its layer range, under a persistent node key. The coordinator verifies every signature and checks that the attested layer blocks tile the model’s **true** depth (62 layers, read from the model configuration, never from the receipts themselves) with no gaps or overlaps — a stage that skips layers, or a ring that omits a stage, fails closed. Receipts for every benchmark in §6 are committed to the repository.

3.5 Fault tolerance

The ring survives the failures a permissionless network guarantees: a dead coordinator is replaced mid-session without re-warming stages (the tail holds its predecessor link and warm KV, adopting the next coordinator’s return channel); a dead stage triggers a cascade re-handshake in which warm stages rebuild forward links without reloading weights. §5.6 reports measured timelines.

4 Speculative decoding over WAN: the law and the design

4.1 The latency wall

A decode step cannot leave the ring faster than one traversal: $T \approx 300\text{--}450$ ms on a good five-stage European ring (RTT floor ≈ 105 ms plus codec, relay and stage time). Autoregressive decoding therefore caps at $1/T \approx 2\text{--}5$ tokens/s regardless of GPU speed — the measured AR baseline in §5.2. Throughput is g/T , where g is committed tokens per traversal. Everything that follows is about raising g (speculation) or hiding T (pipelining).

4.2 The accept-gated pipelining law

Pipelining — keeping D speculative chunks in flight — is the classic answer to latency. For speculative chunks it fails in a specific, quantifiable way: chunk $N+1$ is drafted assuming every token of chunk N commits. With per-token acceptance α and chunk length K , that event has probability α^K ; any rejection flushes the pipe. Expected progress per traversal interval degrades from the pipelined ideal toward the serial rate as $\alpha^K \rightarrow 0$:

- at $\alpha = 0.97$ (verbatim/copyable text, n-gram drafter): $\alpha^8 \approx 0.78$ — pipelining pays, and the system reaches 70–87 tokens/s on such spans;
- at $\alpha = 0.74$ (the strongest published novel-text drafter, EAGLE-3): $\alpha^8 \approx 0.09$ — 91% of traversals flush, and depth buys almost nothing.

The crossover sits near $\alpha \approx 0.80$. We validated this law two ways: a Monte-Carlo simulation of the production pipeline calibrated at both ends reproduces our measured novel-text and verbatim throughputs from α alone, and the live A/B in §5.2 confirms depth is worthless on reasoning cells and decisive on draftable ones. The law has a blunt corollary: **no amount of engineering pipelines novel-text reasoning through a high-latency ring** — only raising α (a drafter-training problem) or cutting T (a transport problem) moves that number.

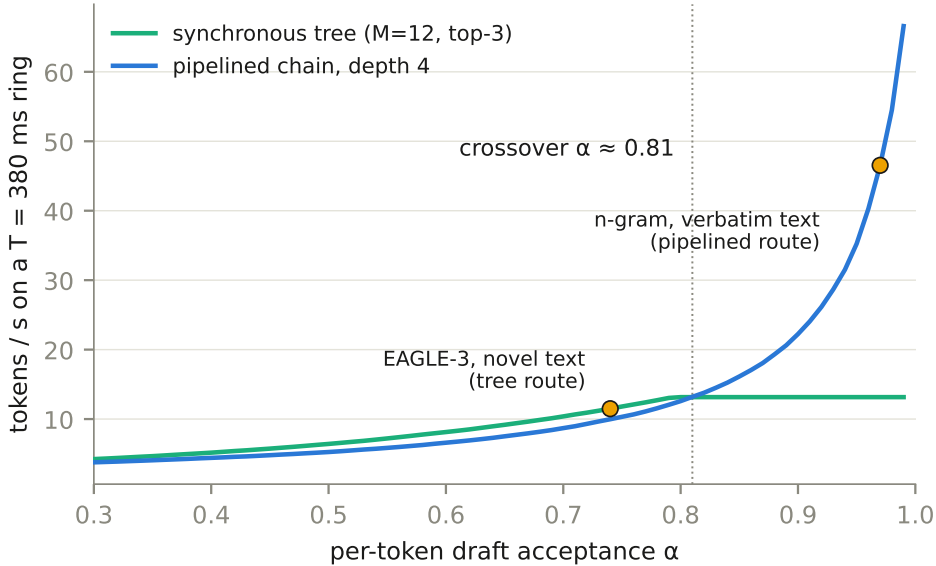


Figure 1: The accept-gated pipelining law: seeded Monte-Carlo of the production pipeline (flush-on-divergence, depth 4) vs the synchronous tree round, calibrated to both measured operating points. Below the crossover, keeping speculative chunks in flight buys nothing — the regime every reasoning workload lives in.

4.3 The depth-aware hybrid

The law dictates a router, not a single strategy. Per round, the coordinator asks its cheap n-gram drafter whether the immediate continuation is **draftable** (a verbatim or near-verbatim span: quoting, copying, code echoes, structured boilerplate):

- **Draftable** → **pipelined linear speculation**. K-token chains framed as plain verification requests, up to D in flight, flash-attention path on every stage, minimal payload. Divergence discards in-flight chunks and re-anchors — bookkeeping identical to classic speculative pipelining.
- **Novel** → **synchronous tree speculation**. An EAGLE-3 head (conditioned on target hidden states returned over the wire as fp8 side-channel tensors) grows a best-first token tree (top-M nodes, branching capped, depth capped); the ring verifies the whole tree in one forward pass under an ancestor-only attention mask, and the longest accepted root-path plus one correction token commits. Trees raise g per (expensive) round-trip precisely where pipelining cannot help.

The two modes interleave freely; a small KV bookkeeping contract (a tree round leaves its committed path’s KV rows dirty; the next frame re-feeds them as a causal prefix) makes the interleaving exact. The entire coordinator is regression-gated by a CPU-only harness that replays a teacher-forced oracle ring over real sockets and asserts token-exact losslessness, KV-frontier integrity, and drafter-pairing invariants across divergence and mode switches — 64 tests, no GPU required.

5 Evaluation

5.1 Methodology

All arms run on **one warm ring** (the five-stage European ring of §3), interleaved cell-by-cell with arm order rotated per repetition, so wide-area and co-tenant drift — which we measured at up to $1.32\times$ across two hours on the same ring — affects every arm equally. Greedy decoding, reasoning enabled, receipts enabled unless stated. Cells cover reasoning, open chat, code editing, retrieval-quoting, tool calling, and document QA at 8k and 30k context. Every job’s full metrics (including per-stage timing and receipt verification) are in the repository as JSON; tables report medians over repetitions with min–max ranges.

5.2 Single-stream results

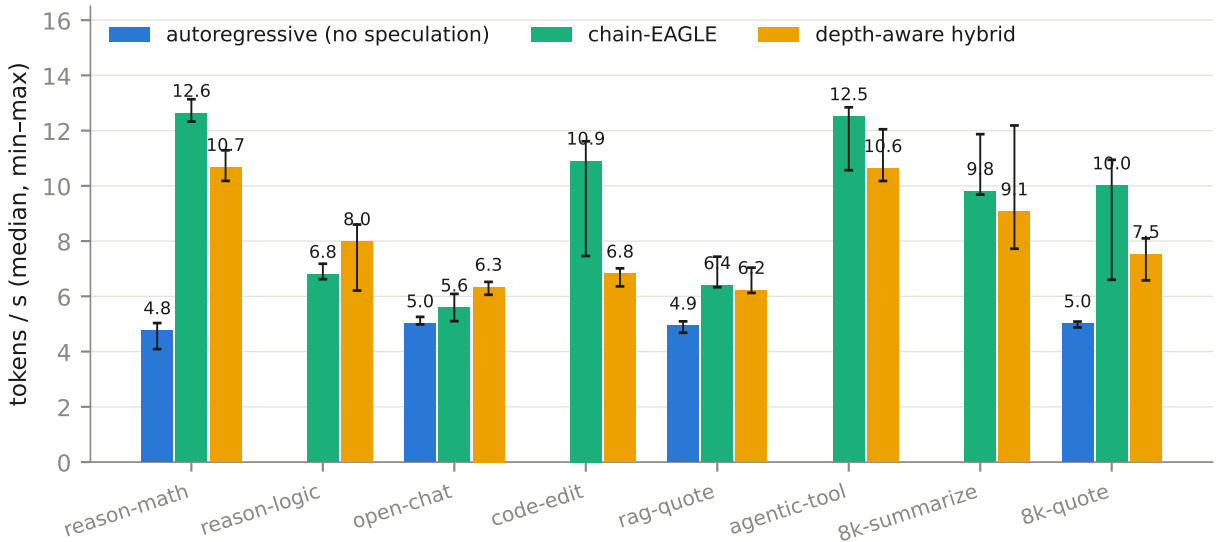


Figure 2: Interleaved three-arm benchmark on one warm ring (medians over 2–3 repetitions, whiskers min–max; receipt `m25-paper-bench-20260703.json`, all 64 jobs receipt-verified).

The autoregressive baseline measures the latency wall directly: **4.8–5.0 tokens/s on every cell**, with exactly $g = 1.00$ committed tokens per traversal by construction — no drafter, no workload dependence, pure g/T physics. Speculative decoding lifts every cell above it. On interactive novel-text cells the chain-EAGLE and hybrid arms reach **10.7–12.6 tokens/s median** (reason-math 12.6 [12.3–13.1] chain, 10.7 [10.2–11.3] hybrid; tool-calling 12.5/10.6; reasoning-logic 8.0 hybrid vs 6.8 chain), a $2.1\text{--}2.7\times$ speedup at identical outputs. On **draftable** spans the pipelined route changes regime entirely: continuing verbatim-

structured text runs at **70.7–87.2 tokens/s single-stream** (receipt `m25-paper-ctxtable-20260703.txt`) — 14–17× the AR wall, exactly the $\alpha \approx 0.97$ branch of the law. A realistic “repeat this document” request that reasons first and then copies lands between the regimes (16.3 tokens/s median, hybrid arm).

Two arm-level observations match the law’s fine print. First, hybrid and chain behave **identically on novel cells** (same g to two decimals: the router sends the same tree rounds), so their differences there are pure ring noise. Second, the tree-vs-chain preference is **ring-speed-dependent**: a tree round carries a fixed compute/payload surcharge over a chain round, so on an unusually fast ring window ($T \approx 250$ ms) the chain arm’s lighter rounds win some novel cells despite lower g , while on slower windows ($T \approx 400$ ms, the common case) the tree’s higher g dominates. A T -aware router is an obvious refinement.

5.3 Where a traversal goes: transport vs compute

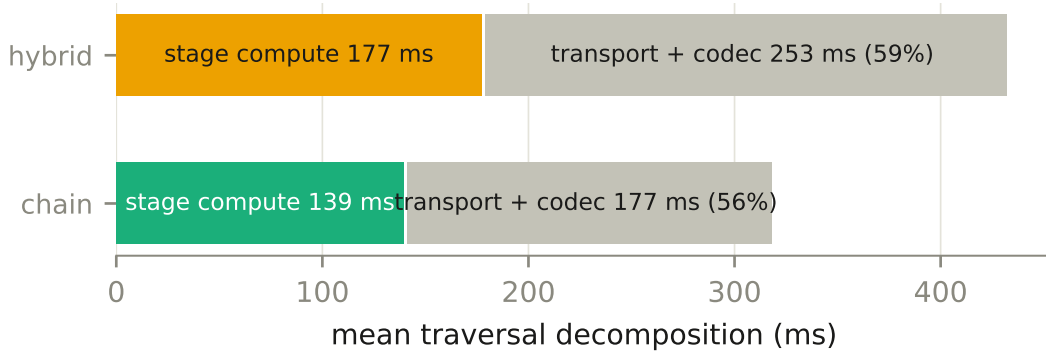


Figure 3: Mean traversal decomposition from per-stage timing stamps (`M25_STAGE_TIMING`), paper-bench cells.

Per-stage stamps decompose every traversal into per-stage spans and a transport remainder (wire + relay + codec). On this ring transport is **55–68% of a traversal** on reasoning cells — against a 105 ms pure-RTT floor, roughly 65–185 ms is addressable overhead (codec and relay), which is the next engineering lever. The compute term produced the report’s most consequential systems finding: **stage time on consumer fleets is CPU-kernel-launch-bound, not GPU-bound**. All five GPUs benchmark identically (1523–1527 GB/s memory bandwidth, 218–227 bf16 TFLOPS measured in situ), yet identical 13-layer blocks take 11.5 ms on stages whose host is an idle desktop CPU (Core Ultra 9/5, 0.09–0.12 s single-thread probe) and 35–50 ms on old or co-tenant-loaded EPYC slices (0.28–0.47 s probe; one candidate box ran at load average 272). A transformer block forward is hundreds of small kernel launches; launch cost is the host’s single-thread speed. Consequences: node selection must probe CPU and load (our ring planner now does), and CUDA-graph capture — dismissed as a 1.05× lever when measured on a fast-CPU box — recovers 2–4× of block time precisely on the slow-CPU boxes a permissionless network actually gets.

5.4 Context axis and batching

Document-QA cells hold usable speeds at real context: 9.8–11.2 tokens/s at 8k context (summarize, chain/hybrid), 10.0–10.8 (quote), and 5.0–6.6 at 30k, where the synchronous tree round’s attention over the full context becomes the bottleneck (phase-1/2 receipt JSONs). Prefill of a 30k-token document takes 22–45 s depending on ring window — time-to-first-token, not decode, is the long-context cost.

Batching amortizes the WAN cost a single stream must eat. On this five-stage ring, $B = 4$ concurrent streams of the draftable continuation task aggregate **150–194 tokens/s** (37–49 per stream) against 71–87 single-stream — a 2.1–2.5× aggregate multiplier at verified per-stream coherence (receipt `m25-paper-ctxtable-20260703.txt`); an earlier six-stage ring sustained **155 tokens/s aggregate at 16.4k context** (receipt, 2026-06-29). One systems constraint surfaced: batch KV competes with stage fatness — our 13-layer tail (weights + KV + the language-model head) capped $B = 4$ context near 12k where the six-stage ring’s lighter tail reached 16k. Stage sizing and batch capacity trade off; the topology planner can optimize for either.

5.5 The cost of verification

Receipts ride every request in every benchmark above (64/64 phase-1 jobs receipt-verified, fail-closed). Their cost, isolated two ways: the per-stage span on drift-free idle-CPU stages is **11.72 ms with receipts vs 11.67 ms without (+0.05 ms, $\approx 0.4\%$)** — the activation hash-chain is a sub-millisecond digest per verification round — while the end-to-end throughput difference between the receipts-on and receipts-off phases (9–16%) is fully explained by measured WAN drift between the two runs, not by verification (the span data bounds the true cost two orders of magnitude below it). Verifiable execution at this granularity is, operationally, free.

5.6 Fault tolerance, live

From the timestamped demo (receipt `m25-paper-ftdemo-20260703.txt`), one continuous timeline on the warm ring: **t = 0 s** — baseline job completes at 14.7 tokens/s. **t = 14 s** — the coordinator is killed with **SIGKILL** mid-decode, four speculative chunks in flight. **t = 34 s** — a brand-new coordinator process connects to the same ring (the tail keeps its warm KV and predecessor link, adopts the new return channel, and drops the dead job’s in-flight frames). **t = 48 s** — the new coordinator has already completed a full 49-token job, prefill included: **zero re-warm, no stage restart, no weight reload**. **t = 60 s** — a receipts-enabled job passes signature and 62-layer coverage verification on the recovered ring. The stage-death path (every warm stage rebuilds its forward link, no weight reload) is exercised by the same harness; a cold stage relaunch — minutes of weight loading — is needed only when a GPU actually disappears, and a warm spare can adopt its layer range instead.

6 Limitations and roadmap

The single-stream ceiling is physics, and we operate near it. The law of §4.2 bounds serial novel-text decoding by g/T with g capped by drafter acceptance; on this ring class that ceiling is roughly 12–14 tokens/s and this report measures 10–13. The remaining levers are engineering (CPU-agnostic stages via CUDA graphs, a leaner wire codec) and one research bet (drafter training toward $\alpha \approx 0.80$, which would also unlock pipelining). Batched serving does not share the ceiling: WAN cost amortizes across streams.

Regional rings. Results are for intra-European rings (5–38 ms legs). Transcontinental rings multiply T ; the same fabric should instead compose regional rings.

Right-sized models. A 229B model needs five hops; a 30–70B model needs one or two, with proportionally higher single-stream speed on the identical fabric. The engine is model-agnostic; a multi-model catalog is the natural next deployment.

Trust scope. Receipts prove layer coverage and bind activations to stages; they do not yet bind to wall-clock or prevent replay across requests. Freshness binding and randomized spot-recomputation are the next verification layers.

7 Conclusion

A 229B mixture-of-experts model serves a single user at interactive speed from five consumer GPUs in four countries that have never met, every request arriving with a cryptographic proof that each stage did its share. The design was not found by scaling intuition but by measurement: a pipelining law that says when speculation can hide the ocean and when it cannot, a timing decomposition that says the bottleneck on consumer fleets is the CPU next to the GPU, and a benchmark discipline that publishes the receipt for every number. That is, we believe, what the permissionless compute thesis needs to be credible — not projections, but a ring you can rent, measure, and verify today.

Reproducibility every table cell links to a signed receipt JSON under `docs/receipts/`; benchmark harnesses are `research/m25_paper_bench.py` and `research/m25_usability_report.py`; the CPU-only correctness harness is `tests/`.

Bibliography

- [1] Dolphin AI, “Distributed Model Sharding,” 2026.
- [2] A. Borzunov *et al.*, “Petals: Collaborative Inference and Fine-tuning of Large Models,” 2022.
- [3] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [4] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023.
- [5] T. Cai *et al.*, “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads,” 2024.
- [6] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [7] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test,” 2025.
- [8] X. Miao *et al.*, “SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification,” in *Proceedings of ASPLOS 2024*, 2024.